

Starclash — Playground integration

INDEPENDENT AUDIT: SOUND

PR #197 · branch `app/game-starclash` · task `application/tasks/game-starclash`

Generated 2026-07-18

What this PR does

Starclash is a crew game (2–4 players share one room, move, chat, and duel at Rock-Paper-Scissors for stars). The Playground samples **one persona per trial**, which doesn't fill a table on its own. This PR makes the crew game runnable as a single-persona Playground/Harbor web task by adding a **player-vs-bots** mode, plus a real vision/computer-use brain, a natural end-state, and per-persona batch reporting.

⚠ **Not a real multi-agent game yet.** In a trial, the one sampled persona is the real player; **every other seat is a computer bot** (`BayesianBotBrain`), not a second sampled persona. So a trial measures *one* persona's play against fixed bot opponents. Hosting several *sampled* personas together in one live match (true multi-agent) requires a **Playground orchestration change** — one trial that runs N persona agents in a shared game — which is **not implemented here**. Comparing persona styles today is done *across separate sessions* (swap the crew manifest), not within one live game.

Feature 1 — Player-vs-bots (the core of this PR)

Piece	What it does
<code>brains.py: BayesianBotBrain</code>	No-API-key, seeded (deterministic) computer opponent. Reads the public <code>arena_card_counts</code> , subtracts its own hand to estimate what opponents still hold, plays the expected-value-maximising counter card, accepts/declines duels by EV, and actively closes distance so duels happen.
<code>run_arena.py router</code>	New <code>--player-id / --opponent-brain</code> . The named seat uses the real <code>--brain</code> ; every other seat becomes a bot with a distinct seed. <code>--player-id auto</code> = first crew persona. Omitting it keeps the original all-brain behaviour (backward compatible).
Env + launch wiring	<code>task.toml [solution.env] → solve.sh → run_arena.py</code> . The launch service (<code>harbor_job_service._starclash_player_id</code>) exports the sampled persona as the player only when it is actually in the crew manifest ; otherwise falls back to <code>auto</code> . Returns <code>None</code> for every non-Starclash task, so nothing else is affected.
Default table	Default <code>crew_manifest.yaml</code> trimmed to 4 personas → a default trial is 1 player + 3 bots = a full 4-seat table (game max). Pinned by a test.

```
MATRIX_STARCLASH_PLAYER_ID (launch service: sampled persona if in crew, else unset)
└─▶ task.toml [solution.env] STARCLASH_PLAYER_ID = ${MATRIX_STARCLASH_PLAYER_ID:-auto}
    └─▶ solve.sh --player-id "${STARCLASH_PLAYER_ID:-auto}"
        └─▶ run_arena.py → player seat = sampled persona · other seats = BayesianBotBrain
```

Feature 2 — Real vision / computer-use brain

`vision_brain.py: BrowserVisionBrain` (`--brain vision`) renders each observation as an HTML HUD, loads it in a **real headless Chromium via Playwright**, screenshots it, and asks Claude (vision + forced tool-use) to click/type. The action is read back from the page's own `window.__lastAction` — genuine browser-use, not a JSON shortcut. Authenticates with a direct `ANTHROPIC_API_KEY` **or** an Amazon Bedrock bearer token (`AWS_BEARER_TOKEN_BEDROCK + AWS_REGION`), defaulting to **Claude Sonnet** (`us.anthropic.claude-sonnet-4-5`) on Bedrock.

Feature 3 — Natural end-state & per-persona reporting

- engine.py:** a game now ends the moment every active persona has emptied its hand (`termination_reason: all_cards_played`) — the intended end state — instead of burning the whole tick budget.
- reporting.json / verifier:** the verifier writes one `task_outcome` + one `task_reasoning_trajectory` context **per crew persona**, grouped by `dominant_trait`, so the batch report can compare outcomes and reasoning style across trait groups.

Verification — live runs I executed

6

battles · live **vision/Sonnet** CUA run (Bedrock, real Chromium, 3m35s)

11/12

player reasoning lines are screenshot-grounded (HUD-aware)

22

ticks → **all_cards_played** oracle 4-seat run (mock + 3 bots)

25

unit tests pass
14 bot + 11 state verifier

Live computer-use run (persona 0229 on `--brain vision` via Bedrock Sonnet + 3 Bayesian bots, seed 42, 16 ticks): **exit 0**, 6 battles, player won with 5 stars, one persona eliminated, valid self-contained `spectator.html` . It took **3m35s** of real API round-trips (not an instant mock), and **11 of 12** of the player's recorded reasoning lines reference visual HUD elements it could only know from a screenshot — e.g. "clicking the MOVE button and selecting a destination tile", "confirming with the center reticle" — confirming genuine screenshot-and-click browser-use (Playwright), distinct from the bots' Bayesian lines and from any JSON fallback.

⚠️ Playground-UI launch on this host. The task's computer-use game logic is validated live (above). Launching the *same* task through the Playground **backend UI** on this Windows machine failed before any trial started with `WinError 2` : the backend's harbor launcher resolves a Unix `.venv/bin/harbor` path and falls back to a bare `harbor` executable not on the Windows `PATH` . This is a **pre-existing backend/environment issue, unrelated to the game-starclash task code** (the task never executed in that path), and is out of scope for this PR. The task *is* correctly discovered and launchable via `/api/web-eval/tasks` ; a Unix/CI host with `harbor` on `PATH` (or `MATRIX_HARBOR_COMMAND` set) runs the full trial path.

Independent audit result

A separate verifier agent adversarially audited every change against the task's own rules. **Verdict: SOUND — no correctness bug.** The highest-risk check passed decisively:

Check	Result
Bayesian posterior double-subtract	PASS — <code>arena_card_counts</code> includes the bot's own cards, so subtracting own hand once is correct (not a bug).
Counter map ↔ engine RPS rules	PASS — <code>_COUNTER</code> is the correct inverse of <code>engine._BEATS</code> .
Observation field names	PASS — every field the bot reads is produced by <code>observations.py</code> (no silent degrade).
Env-wiring precedence & task isolation	PASS — CLI flag beats env; other tasks return <code>None</code> , unaffected.
Natural end-state edge cases	PASS — <code>all_cards_played</code> can't fire on an empty active roster; <code>run()</code> matches <code>is_done()</code> .
Backward compatibility (no <code>--player-id</code>)	PASS — identical to prior all-brain behaviour.
Task ↔ Docker script-copy parity	PASS — all script + input copies byte-identical (the Docker copy is what actually runs).

The one deviation the audit flagged — default roster was 5 seats vs. the intended max of 4 — has been **fixed in this update**: the manifest is trimmed to 4 and a test pins it.

Status summary

Item	Status
One sampled persona plays a full game vs. bots	DONE
Bayesian bots over public card counts, deterministic	DONE
Default 4-seat table (1 player + 3 bots)	DONE
Real vision/CUA brain (Sonnet via Bedrock), live-verified	DONE
Task discoverable/launchable as a Playground web task	DONE
Multiple <i>sampled</i> personas in one live match (true multi-agent)	NOT YET — needs Playground orchestration change